

# Deep Research Agent Swarm

## 50 Backend Interview Questions & Answers

# Deep Research Agent Swarm - 50 Backend Interview Questions & Answers

Scope: Backend (FastAPI, LangGraph, persistence, trust scoring, SSE). Aligned with the backend/ implementation in this repository.

### Q1. What is the primary aim of this project's backend?

The backend runs a hierarchical multi-agent research pipeline end to end: it accepts a user query, executes a LangGraph workflow (plan, parallel search, critic loops, claim extraction, fact-checking, synthesis, citation review), persists session state and an append-only event log in Postgres (plus LangGraph checkpoints for resume), exposes REST for lifecycle control, and streams Server-Sent Events so the UI can show live agent activity. A distinctive product goal is auditable confidence: numeric trust scores and citation-alignment checks, not only a polished prose report.

### Q2. What business problem does a multi-agent "swarm" solve that a single long chat agent does not?

Three bottlenecks: (1) context pollution - one agent accumulates noisy tool dumps until quality collapses; workers return short summaries plus source IDs. (2) lack of specialization - one "do everything" prompt is weaker than planner + search + critic + synthesizer roles. (3) no internal adversary - confident hallucination; a Critic and Fact-Checker add explicit self-correction. \*Example:\* fifteen raw search results in one thread vs five workers each summarizing one sub-question in isolation.

### Q3. Why use an orchestrator-worker pattern instead of a peer-to-peer agent mesh?

Determinism and debuggability: one coordinating path (the graph) decides what runs next; workers consume a task and return a structured result. A free-for-all mesh (common in some multi-agent demos) burns tokens on coordination, is hard to replay, and obscures accountability. Alternative: peer networks (e.g., conversational agent loops) when exploration beats audit trails - poor fit for regulated-style research output.

### Q4. Why was LangGraph chosen over CrewAI, AutoGen, or a hand-rolled DAG?

LangGraph gives first-class cycles (critic -> search again), typed graph state with reducers, and an official Postgres checkpoint for resume. CrewAI is faster for linear role stories but awkward for bounded feedback loops at scale. AutoGen-style chatter is often non-hierarchical. Alternative: Temporal.io or Prefect for workflow durability - heavier ops unless you already run them.

### Q5. Why implement graph nodes as plain async Python functions instead of LangGraph's higher-level agent wrappers?

Full control over prompts, tool rails, budgeting, telemetry, and error handling. Framework "agents" hide composition and couple you to generic tool loops. This codebase uses `wrap_node` for observability while keeping business logic in normal functions.

### Q6. Describe the compiled LangGraph topology in this repository.

From `app/graph/build.py`: `START -> planner -> parallel_search -> critic`. A conditional edge runs `after_critic`: either `search_again` (via `critic_route_prepare` back to `parallel_search`) or `extract_claims`. Then `fact_check -> synthesize -> citation_format -> END`.

### Q7. How is the critic -> search loop prevented from running forever?

`after_critic` in `app/graph/nodes/router.py` only routes to `search_again` if there are follow-ups and `critic_round < max_critic_rounds` (settings). `critic_route_prepare_node` increments the round, clears follow-ups, and enqueues new `pending_sub_questions`. Alternative policy: budget-based stopping (already complemented by session cost caps).

### Q8. What is ResearchGraphState, and why use Annotated[..., operator.add] fields?

Defined in `app/schemas/state.py`: the shared TypedDict for all nodes. Lists like `findings_summaries`, `all_sources`, and `messages` use `operator.add` reducers so partial updates from parallel branches merge safely; `cost_usd` uses a custom summing reducer `add_cost`. Example: two nodes appending sources cannot overwrite each other's lists - the graph merges updates.

### Q9. How does Postgres support both product data and LangGraph resume?

Application tables hold `research_sessions` and `research_events`. `AsyncPostgresSaver` (see `research_runner.py`) stores checkpoints in dedicated tables using a `thread_id` keyed by session UUID, so `resume=True` continues from the last node transition. Alternative: separate DB cluster for checkpoints if you want noisy workflow I/O isolation.

**Q10. What does `run_research_job` do with `astream` / `ainvoke`, and why does it matter for the UI?**

It compiles the graph with the checkpoint, runs it with `thread_id=str(session_id)`, and on stream ticks can emit `cost_update` events and sync `total_cost_usd` / `agent_invocation_count` to the session row - so the dashboard cost ticker stays fresh even though long LLM calls show zero invocations until a node completes.

**Q11. Walk through the five trust dimensions and their weights in production code.**

In `compute_trust_breakdown` (`app/services/trust.py`): `source_count`, `source_authority` (mean domain heuristic), `source_agreement` (placeholder), `recency`, and `fact_checker` (verifier score). Weights are 0.15, 0.25, 0.15, 0.15, 0.30. The headline `trust_score` is the rounded, clamped weighted sum. \*See also\* `docs/backend/04-scoring-trust-and-similarity.md`.

**Q12. What is the exact source-count formula when sources exist?**

For `n` supporting catalog sources: `c_count = min(100, 35 + 15 * max(0, n - 1))`. One source starts at 35; each additional independent source adds up to 15 points, capped at 100. \*Example:\* `n=3 -> 35+30=65`.

**Q13. How is `source_authority` computed from a URL?**

`_domain_authority` buckets the hostname: e.g. `.gov/.mil` 95, `.edu` 88, `wikipedia.org` 85, `github.com/arxiv.org` 80, `generic.org` 65, default 55, parse problems 30. The claim score is the mean across attached sources. Alternative: integrate a third-party domain reputation API or crawl-derived PageRank-style features.

**Q14. Why is `source_agreement` fixed at 70 today? What ML technique could replace it?**

It is a neutral placeholder until a real agreement signal ships - avoids bundling an entailment model in the OSS baseline. Alternatives: NLI / textual entailment (e.g., DeBERTa cross-encoder) comparing each snippet to the claim; embedding variance across sources; LLM-as-judge consistency score (costlier, drift-prone).

**Q15. How does recency scoring work numerically?**

Missing or bad dates -> 60 (neutral). Else compute age in days `*d*` and per-source score clamped to `[20,100]`: roughly 100 when fresh, trending to ~20 near 5 years (1825 days factor). Claim recency is the average across sources. Example: one-week-old article scores near 100; a 10-year-old paper drags the average down for time-sensitive claims.

**Q16. Where does the `fact-checker` numeric score come from?**

`fact_check_node` runs a fresh verification search and asks `model_fast` for JSON including a 0-100 score; missing/invalid values default to 50 in the trust pipeline. That `f` becomes the `fact_checker` dimension (or 55 default in the weighted blend if absent in some paths - see trust merge code vs verifier injection in `postprocess.py`).

**Q17. What special-case logic applies when a claim has no resolved sources?**

`compute_trust_breakdown` short-circuits: no weighted blend; headline trust is capped via `**round(f * 0.45)` clipped to ?45 so provenance cannot be papered over\*\* by a generous verifier alone.

**Q18. Why give `fact_checker` the largest weight (30%)?**

It reflects independent retrieval and reasoning on the claim - less brittle than URL heuristics alone (typosquat domains, SEO spam). It still isn't ground truth; it complements evidence count and authority.

**Q19. How does citation alignment detect "hallucinated" citations after synthesis?**

`citation_formatter_node` scores similarity between each claim and each cited basis (`full_content[:4000]` preferred else snippet). It takes the minimum across citations; if `min < CITATION_SIMILARITY_THRESHOLD` (default 0.7), the claim gets `LOW_CITATION_ALIGNMENT`. Weakest-link semantics: one bad supporting link fails the chain.

**Q20. Compare TF-IDF cosine vs embedding similarity for citation checking in this codebase.**

Default `similarity_mode=tfidf`: cheap, local, deterministic - no extra API; weaker on paraphrase. `similarity_mode=litellm`: calls an embedding model - better semantic match, adds latency/cost. \*Example:\* paraphrased claim may score low on bag-of-words but high on embeddings.

**Q21. How is SSE implemented in FastAPI for live sessions?**

Route `GET /api/v1/research/{session_id}/stream` returns `StreamingResponse` with `media_type="text/event-stream"` and `Cache-Control: no-cache`, `Connection: keep-alive`, `X-Accel-Buffering: no`. Generator `sse_event_stream` polls `research_events` for rows with `id > after_id`, yields data: `{json}\n\n`, sleeps `SSE_POLL_INTERVAL_SECONDS`, and terminates after a terminal `session_status` when the session completes or fails.

**Q22. Why poll Postgres for SSE instead of pushing from an in-memory queue?**

Multi-worker safety - any API replica can serve the stream; ordering relies on monotonic `research_events.id`. Tradeoff: sub-second latency vs complexity of Redis Pub/Sub or LISTEN/NOTIFY. Alternative: Redis Streams with consumer groups for lower latency fan-out.

**Q23. Explain after\_id reconnect semantics.**

Clients track the largest event id they have processed; on reconnect they call ?after\_id=N so the server replays only newer rows, closing gaps after dropped connections without duplicating the entire history.

**Q24. What kinds of events are written to research\_events, and why durable storage?**

Examples from architecture docs: agent\_started, tool\_call, claim\_verified, cost\_update, session\_status. Durability decouples graph execution from transport - refreshes, reconnects, and audits all see the same canonical log.

**Q25. How is unconstrained LLM concurrency limited?**

app/services/llm.py uses a global asyncio.Semaphore from max\_parallel\_agent\_calls to cap simultaneous completions, reducing provider rate-limit storms.

**Q26. How does budget\_exceeded protect sessions?**

app/graph/nodes/\_util.py compares state["cost\_usd"] to session\_cost\_limit\_usd; when true, nodes should skip expensive work. Example: kill switch at \$5 prevents runaway spend on pathological queries.

**Q27. What does wrap\_untrusted do for security?**

It fences web-derived excerpts in <untrusted\_web\_content>...</untrusted\_web\_content> so prompts frame them as data, not instructions. This reduces (not eliminates) prompt injection risk from malicious pages. Alternative: separate retrieval channel with strict output schema + allowlisted domains.

**Q28. How are tool rails chosen per sub-question?**

tools\_for\_subq reads tools or assigned\_tools from planner output, default ["search\_web"], capped to four rails to limit prompt size and tool-choice confusion.

**Q29. What is the unified tool output philosophy?**

Tools normalize to SourceDict (source\_id, url, title, snippet, optional full\_content, published\_date, domain\_authority, tool\_name) so downstream agents reason uniformly regardless of Tavily, GitHub, arXiv, etc. Alternative: returning raw JSON blobs per provider - hurts composability.

**Q30. Why separate synthesizer and citation\_formatter nodes?**

Different failure modes: synthesis needs narrative integration; citation formatting enforces link supportability and bibliography structure - often callable with a faster model tier in a cost-optimized design.

**Q31. What ML / LLM techniques are used, and why those choices?**

LLM structured JSON for plans, claims, verification scores; TF-IDF or embeddings for citation-source alignment; heuristic trust for interpretability. Why: balance cost, latency, and auditability. Alternatives: end-to-end seq2seq without explicit claims (opaque); heavyweight entailment everywhere (accurate but slow/expensive).

**Q32. If you removed LangGraph tomorrow, what would you replace it with?**

A deterministic state machine (explicit enums + transitions) with your own checkpoint serializer, or Temporal workflows for long-running durability. You would reimplement cycle handling and merge reducers carefully.

**Q33. How would you calibrate trust scores against labeled data?**

Collect claims with human truth labels; bin predicted trust\_score; plot reliability; fit Platt scaling or isotonic regression on a dev set; tune weights so scores >80 exceed your target precision (e.g., >85% correct). Example metric: Brier score or ECE for probabilistic audit story.

**Q34. What's the difference between claims and verified\_claims in graph state?**

claims are extracted structures; after fact-checking and trust attachment they populate verified\_claims (with trust\_score / trust\_breakdown) for synthesis - synthesis should not depend on unverified raw tool transcripts.

**Q35. Why use LiteLLM instead of direct Anthropic/OpenAI SDKs?**

Provider abstraction, shared completion paths, and built-in hooks (e.g., Langfuse callbacks in observability.py). Lets you fail over models without rewriting nodes. Alternative: individual SDKs behind an internal adapter interface - viable but more glue code.

**Q36. How does model routing save money in this architecture (conceptually)?**

Strong models on planner/critic/synthesis; fast models on search summarization and citation formatting. Example: ten cheap Haiku-class calls plus two Sonnet-class calls beats ten Sonnet-class search loops.

**Q37. What failure mode does "Critic-Searcher infinite loop" describe, and how is it addressed?**

The critic forever requests more research. Fix: hard max\_critic\_rounds cap - after the cap, downstream proceeds and low-trust scoring surfaces weak evidence instead of blocking forever.

**Q38. What failure mode does "source hallucination" describe, and how is it addressed?**

The synthesizer cites a URL that does not support the claim. Fix: citation similarity gate + explicit flags in the report so the UI can highlight misalignment.

**Q39. How does checkpoint thread\_id = str(session\_uuid) help operators?**

One stable key ties all checkpoints for a session; resume endpoints can reload exactly where the graph stopped - important after worker crashes or deployments.

**Q40. Why keep single Postgres instead of Redis + Postgres split (as some specs suggest)?**

Operational simplicity: one backup, one migration story, transactional story with events + sessions. Tradeoff: SSE polling adds DB read load versus sub-ms Redis reads. Scale-out path: add read replicas or introduce Redis for hot counters.

**Q41. How would you evaluate "40% accuracy lift vs single-agent baseline" credibly?**

Curate a fixed N-question dataset; run single long-context agent vs swarm with frozen tool access; blind human or rubric-based scoring for correctness and completeness; publish methodology and confidence intervals. Instrument actual cost\_update for fair cost comparisons.

**Q42. What role does messages in state play?**

An append-only human-readable trace (Annotated[list[str], operator.add]) for debugging and coarse-grained narrative of graph progress alongside structured fields.

**Q43. How does \*\*critic\_route\_prepare\_node keep routing "pure" in LangGraph?**

Conditional edge functions should avoid mutating state; the helper node materializes pending\_sub\_questions and increments critic\_round, keeping transition predicates side-effect free.

**Q44. Why might domain\_authority heuristics misfire, and what's the mitigation?**

Good content on unknown domains scores middling (55); governments with misleading pages still score high heuristically. Mitigations: red-team lists, manual blacklist, blend with fresh verification (fact\_checker weight), and user warnings in UI bands (e.g., 51-80 "moderate").

**Q45. How does RAG over a private corpus differ from this open-web swarm?**

RAG excels when answers live in controlled embeddings; this system targets evolving web research with adversarial critique and explicit citations. Hybrid: internal pgvector tool rail for org docs plus external search workers.

**Q46. What API concerns matter when deploying SSE behind nginx or Kubernetes ingress?**

Disable proxy buffering (X-Accel-Buffering: no), ensure idle timeouts exceed expected run length, and size workers knowing each SSE holds a long connection.

**Q47. How does schema evolution interact with LangGraph checkpoints?**

Changing ResearchGraphState shape can invalidate old checkpoints or require migration defaults. Mitigation: additive fields, default reducers, and version pins in run\_config events for reproducibility.

**Q48. What's a concrete alternative trust model to hand-tuned weights?**

Train a gradient boosted tree or small MLP on dimension features + textual embeddings; or Learning-to-Rank from editorial judgments. Tradeoff: gains accuracy but loses instant stakeholder explainability unless you add SHAP.

**Q49. Name a simplification in the current trust stack an interviewer might probe.**

source\_agreement constant - honest interview answer: "placeholder until entailment ships; today agreement is assumed neutral-high." Pair with roadmap: NLI cross-encoder or multi-snippet clustering.

**Q50. Summarize the backend's "resume metrics" story in one interview sound bite.**

"We meter USD and invocations per session (cost\_update + DB counters), cap critic rounds and session cost, stream durable SSE with after\_id, and expose 0-100 trust with per-dimension breakdown plus citation similarity flags - so accuracy, cost, and provenance are all inspectable, not just final prose."

---

\*Generated for interview preparation. Cross-check behavior in backend/ and docs/backend/ as the code evolves.\*